ΑΠΟ ΤΑ ΛΟΓΙΚΑ ΔΙΑΓΡΑΜΜΑΤΑ

ΣΤΗΝ ΨΕΥΔΟΓΛΩΣΣΑ

ΚΑΙ

ΑΠΟ ΤΗΝ ΨΕΥΔΟΓΛΩΣΣΑ

ΣΤΗΝ ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C

Μ.Ι.Σ. 2021-2022

Η μέχρι σήμερα, διαδικασία περιγραφής των αλγορίθμων μας, αφορούσε κυρίως το λογικό διάγραμμα και την ψευδογλώσσα (ή ψευδοκώδικα).

Τα λογικά διαγράμματα και ο ψευδοκώδικας δεν μπορούν να διαβαστούν από τους Η/Υ καθώς δεν έχουν αυστηρούς συντακτικούς κανόνες που να γίνονται κατανοητοί από τους Η/Υ και χρησιμοποιούνται κυρίως για την αποτύπωση-περιγραφή-κατανόηση των προβλημάτων μεταξύ ανθρώπων.

Οι γλώσσες προγραμματισμού αντίθετα, έχουν πολύ αυστηρό συντακτικό –ΣΥΓΚΕΚΡΙΜΕΝΟΥΣ ΚΑΝΟΝΕΣ- (που περιορίζουν τον άνθρωπο-προγραμματιστή να τους σεβαστεί) ώστε να είναι ερμηνεύσιμο και κατανοητό από μια μηχανή.

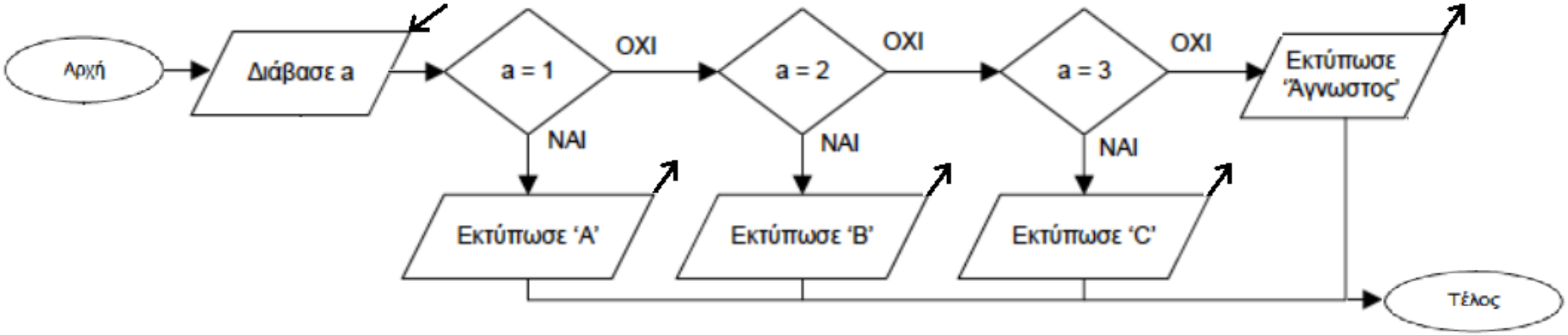
Ένα πρόγραμμα συνεπώς είναι ένα σύνολο από ΣΥΓΚΕΚΡΙΜΕΝΕΣ ΟΔΗΓΙΕΣ που μπορούν να εκτελεστούν μηχανικά από τον Η/Υ, χωρίς να απαιτούν κάποια «σκέψη» ή «κρίση» από την μηχανή.

Για παράδειγμα, το επόμενο πρόβλημα,

«Να διαβαστεί ένας ακέραιος και να εκτυπωθεί το αντίστοιχο γράμμα του λατινικού αλφαβήτου, αν ο ακέραιος έχει την τιμή 1 ή 2 ή 3, διαφορετικά να εκτυπωθεί η λέξη 'Άγνωστος'»

Μπορεί να αποτυπωθεί από ένα λογικό διάγραμμα σε ψευδογλώσσα και από ψευδογλώσσα σε γλώσσα προγραμματισμού C ως εξής,

Μ.Ι.Σ. 2021-2022




```
Αλγόριθμος Παράδειγμα
  Διάβασε a
  Αν a = 1 τότε
      Εκτύπωσε "A"
  αλλιώς_αν a = 2 τότε
      Εκτύπωσε "B"
  αλλιώς_αν a = 3 τότε
      Εκτύπωσε "C"
  αλλιώς
      Εκτύπωσε "Άγνωστος"
  τέλος_αν
Τέλος
```

```
#include <stdio.h>
main()
{
    int a;
    clrscr();
    printf("Dose tin timi tou akeraiou:");
    scanf("%d",&a);
    if(a==1)
          printf("A");
    else if(a==2)
          printf("B");
    else if(a==3)
          printf("C");
    else
          printf("Agnwstos!");
    getch();
}
```

ΟΙ ΓΛΩΣΣΕΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ

ΚΑΙ

Η ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C

Μ.Ι.Σ. 2021-2022

Είναι ήδη γνωστό ότι ένας Η/Υ καταλαβαίνει μία και μοναδική γλώσσα, όπου οι οδηγίες και τα δεδομένα αποτυπώνονται με την μορφή δυαδικών ψηφίων, 1010000101001010010011001101010100.

Η επικοινωνία ανθρώπων και Η/Υ είναι ένα ανοικτό πρόβλημα το οποίο μπορεί να λυθεί με δύο τρόπους, ή ο άνθρωπος θα μάθει τη γλώσσα του Η/Υ ή ο Η/Υ να μάθει την ανθρώπινη γλώσσα. Σαν αποτέλεσμα του προηγούμενου συλλογισμού, μπορούμε να θεωρήσουμε ότι υπάρχουν δύο κατηγορίες **γλωσσών προγραμματισμού**, οι γλώσσες που είναι προσανατολισμένες προς την μηχανή και οι γλώσσες που είναι προσανατολισμένες προς τον άνθρωπο.

Οι **γλώσσες προγραμματισμού**, έχουν επινοηθεί σε διάφορες παραλλαγές, με συγκεκριμένες εντολές προς τον Η/Υ χρησιμοποιώντας κάποιους συνδυασμούς λέξεων, φράσεων, προτάσεων, γραμμάτων και αριθμών που απομνημονεύονται από τον άνθρωπο ευκολότερα από τα απλά δυαδικά ψηφία, 0 και 1.

<u>Κατηγορία γλωσσών προσανατολισμένων προς την μηχανή</u>

Αυτού του τύπου οι γλώσσες (Assembly) ανήκουν στη κατηγορία των προσανατολισμένων προς την μηχανή, γιατί στην δημιουργία τους λαμβάνεται υπ' όψιν η συγκεκριμένη μηχανή για την οποία έχουν φτιαχτεί και αγνοείται το πρόβλημα. Οι εντολές είναι συγκεκριμένες για κάθε τμήμα του Hardware. Αφού γραφεί ένα τέτοιο πρόγραμμα, για να γίνει κατανοητό από τον Η/Υ πρέπει πρώτα να μεταφρασθεί σε γλώσσα μηχανής, η μετάφραση γίνεται από ένα ειδικό πρόγραμμα που ονομάζεται Assembler. Η Assembly είναι μια δύσκολη γλώσσα με ειδικές διαφορετικές εντολές για κάθε τύπο Η/Υ και απαιτεί ειδική γνώση του Hardware από τον προγραμματιστή.

<u>Κατηγορία γλωσσών προσανατολισμένων προς τον άνθρωπο</u>

Οι προσανατολισμένες προς τον άνθρωπο γλώσσες, έχουν δημιουργηθεί με σκοπό να έρθουν κοντά στην ανθρώπινη φύση και λογική. Στην γραφή τους χρησιμοποιούνται τα γράμματα της Αγγλικής γλώσσας, οι αριθμοί καθώς και σύμβολα. Το σύνολο των οδηγιών που γράφει ο προγραμματιστής αποτελεί το πηγαίο πρόγραμμα (source program). Στην συνέχεια για να φτιαχτεί το πρόγραμμα που θα εκτελεστεί από τον Η/Υ, χρειάζεται ένα ειδικό πρόγραμμα (compiler ή interpreter) που μεταφράζει το πηγαίο πρόγραμμα σε γλώσσα μηχανής. Έτσι προκύπτει το αντικείμενο πρόγραμμα

Μ.Ι.Σ. 2021-2022

(object program). Το αντικείμενο πρόγραμμα με την βοήθεια του Λ.Σ. (Loader) θα μετατραπεί σε εκτελέσιμο πρόγραμμα (executable program) και θα είναι πλέον έτοιμο να εκτελεσθεί από τον Η/Υ.

Μερικές από τις ποιο γνωστές, προσανατολισμένες προς τον άνθρωπο, γλώσσες προγραμματισμού είναι οι παρακάτω,

FORTRAN (FORmula TRANslation) δημιουργήθηκε το 1955 από την IBM για την επίλυση επιστημονικών και τεχνικών προβλημάτων. Βασικά της πλεονεκτήματα είναι η εύκολη διατύπωση μαθηματικών εκφράσεων και το μικρό πλήθος εντολών. Από το 1955 η FORTRAN βελτιώθηκε πολλές φορές με αποτέλεσμα να έχουμε τις εκδόσεις FORTRAN IV, V, 66 όπως και τις πιο πρόσφατες FORTRAN 77, FORTRAN 88 , Parallel FORTRAN κ.λπ.

COBOL (COmmon Business Oriented Language) δημιουργήθηκε το 1959 και προορίζεται για οικονομικές και εμπορικές εφαρμογές. Αυτά τα προβλήματα παρουσιάζουν μεγάλο όγκο δεδομένων και σχετικά μικρή επεξεργασία (αντίθετα με τα επιστημονικά ή τεχνικά). Χαρακτηριστικό της γλώσσας είναι το πλούσιο λεξιλόγιό της, κάποιος που γνωρίζει απλά αγγλικά μπορεί να «διαβάσει» εύκολα ένα COBOL πρόγραμμα.

ALGOL (ALGOrithmic Language), παρουσιάσθηκε το 1958 σαν αποτέλεσμα μιας διεθνούς συνεργασίας. Προορίζεται για επιστημονικοτεχνικά προβλήματα. Έχει σχεδόν όλες τις ιδιότητες της FORTRAN και μεγάλη ευελιξία στη χρήση της. Οι νέες δομές που πρωτοχρησιμοποιήθηκαν στην ALGOL, είχαν σαν αποτέλεσμα να αλλάξουν τα πρότυπα των γλωσσών και να δημιουργηθούν νέα δεδομένα στη φιλοσοφία τους. Η ALGOL θεωρείται ο πρόγονος σχεδόν όλων των σύγχρονων γλωσσών προγραμματισμού όπως η Pascal και η C .

BASIC (Beginner's All purpose Symbolic Instruction Code) άρχισε να αναπτύσσεται το 1964 στο Dartmouth College των ΗΠΑ από τους J. Kemeny και T. Kurtz. Αρχικά η γλώσσα προορίζονταν καθαρά για εκπαιδευτικούς σκοπούς, γι’ αυτό και πήρε αυτό το όνομα. Αργότερα αναπτύχθηκε και συμπληρώθηκε σε τέτοιο βαθμό που θεωρείται σήμερα σαν μια αρκετά καλή γλώσσα για αρχάριους αλλά και για σοβαρές εφαρμογές. Κύρια χαρακτηριστικά της είναι η υπερβολικά απλή της σύνταξη και το επίσης απλό της λεξιλόγιο. Γενικά, μπορεί να χαρακτηρισθεί σαν ένα υποσύνολο της FORTRAN.

Μ.Σ. 2021-2022

PASCAL οφείλει το όνομά της στο Γάλλο μαθηματικό Blaise Pascal (1623-1662). Δημιουργήθηκε από το N. Wirth το 1971 παίρνοντας πολλά στοιχεία από την ALGOL. Σήμερα θεωρείται η καθεαυτό εκπαιδευτική γλώσσα στην εκπαίδευση της πληροφορικής αλλά χρησιμοποιείται και σε εμπορικές εφαρμογές.

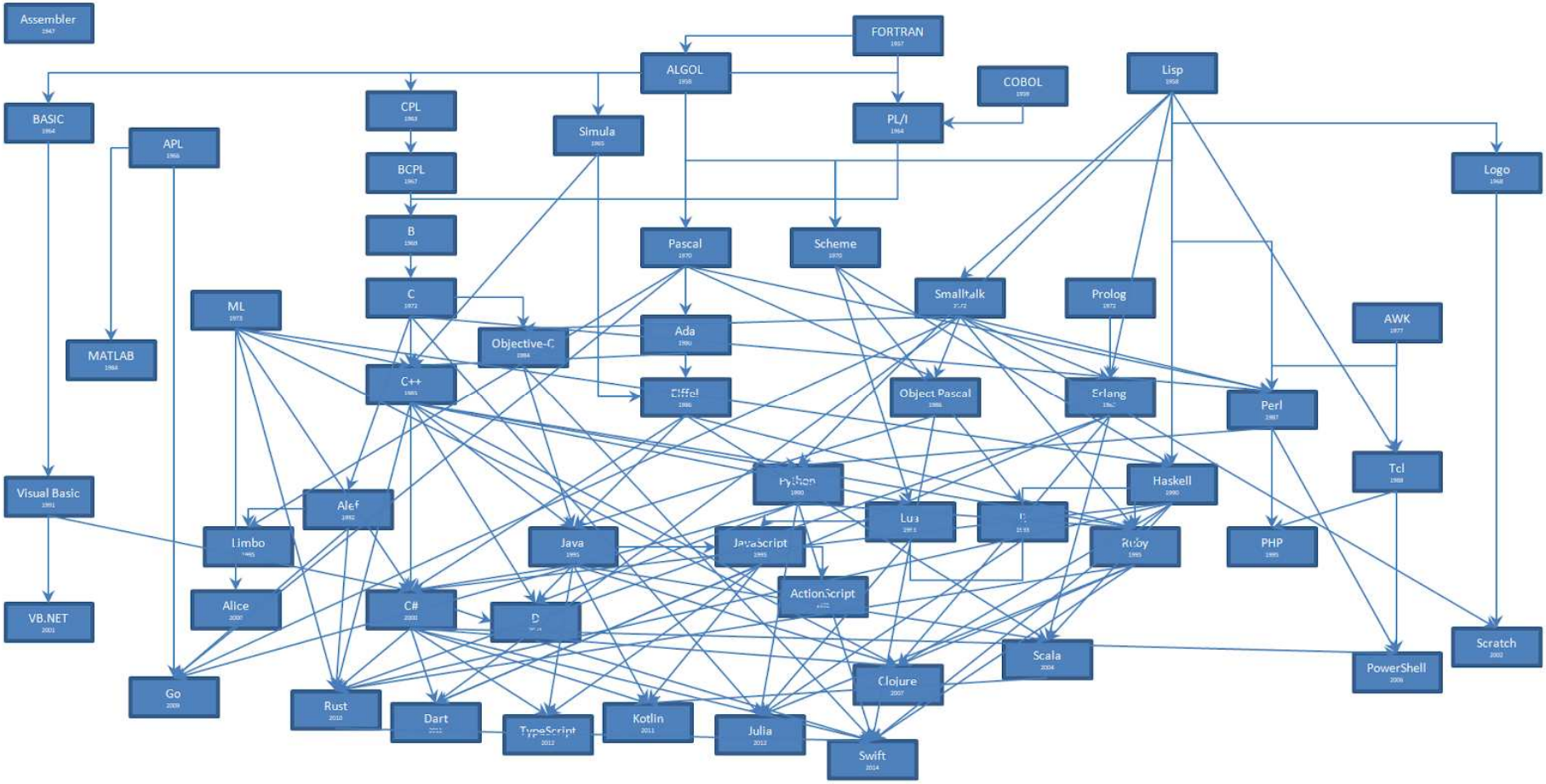
C, δημοφιλής γλώσσα που σχεδιάστηκε από τον D.Ritcie το 1972. Ανήκει στις λεγόμενες αλγοριθμικές γλώσσες ALGOL-like. Βασικά χαρακτηριστικά της είναι το συμπυκνωμένο αλλά δυναμικό λεξιλόγιο της και οι ισχυρές δομές που υποστηρίζει. Σήμερα, ένα μεγάλο ποσοστό του σύγχρονου software είναι γραμμένο σε C.

C++ αποτελεί ένα υπερσύνολο της C και οι επιπλέον δυνατότητες της γλώσσας αυτής είναι ότι επιτρέπει αντικειμενοστραφή (object oriented) προγραμματισμό.

ADA (προς τιμή της Augusta Ada Byron 1815–1851, που ήταν βοηθός του C. Babbage και η κόρη του γνωστού ποιητή Λόρδου Βύρωνα). Δημιουργήθηκε το 1979 για τις ανάγκες του Υπουργείου Άμυνας των ΗΠΑ. Σήμερα χρησιμοποιείται σε ευρύ επίπεδο. Επειδή είναι νέα γλώσσα, αυτή τη στιγμή είναι ακόμη στη φάση της κριτικής, μια φάση που όλες οι γλώσσες έχουν περάσει. Βασικά σχεδιάστηκε για μεγάλα συστήματα ΗΥ και πολλοί επιστήμονες της πληροφορικής διαφωνούν ως προς το αν μπορεί να χρησιμοποιηθεί σε μικρότερους ΗΥ.

Java, μία πολύ σύγχρονη αντικειμενοστραφής γλώσσα προγραμματισμού, αποτελεί την πιο δημοφιλή γλώσσα για προγραμματισμό στο Internet. Είναι μία δυναμική και συνεχώς αναπτυσσόμενη και βελτιούμενη γλώσσα. Ο αντικειμενοστραφής προγραμματισμός (object oriented programing, OOP) ξεκίνησε στην δεκαετία του 1960 (SmallTalk), όπου ένα πρόγραμμα είναι μια συλλογή από «αντικείμενα-υποπρογράμματα» τα οποία επικοινωνούν μεταξύ τους με μηνύματα. Κάθε ένα από αυτά τα αντικείμενα μπορεί να λαμβάνει μηνύματα, να επεξεργάζεται δεδομένα και να στέλνει μηνύματα σε άλλα αντικείμενα. Στην πραγματικότητα, είναι μια διαδικασία ενεργοποίησης μικρότερων μονάδων λογισμικού.

Μ.Σ. 2021-2022


Assembler
FORTRAN
ALGOL
COBOL
Lisp
BASIC
CPL
Simula
PL/I
APL
BCPL
Logo
B
Pascal
Scheme
C
Smalltalk
Prolog
ML
AWK
Ada
Objective-C
MATLAB
C++
Eiffel
Object Pascal
Erlang
Perl
Tcl
Haskell
Visual Basic
Alef
Python
Lua
Limbo
Java
JavaScript
Ruby
PHP
Alice
C#
D
ActionScript
VB.NET
Scala
Scratch
PowerShell
Clojure
Go
Rust
Dart
TypeScript
Kotlin
Julia
Swift

ΔΗΜΙΟΥΡΓΙΑ ΕΝΟΣ ΠΡΟΓΡΑΜΜΑΤΟΣ ΣΕ ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ ΟΙ ΜΕΤΑΦΡΑΣΤΕΣ ΜΕΤΑΓΛΩΤΤΙΣΤΕΣ (COMPILERS, INTERPRETERS)

Για την μετατροπή ενός πηγαίου προγράμματος (source code) σε εκτελέσιμο πρόγραμμα, είναι αναγκαία μια διαδικασία που ονομάζεται compilation (μετάφραση ή μεταγλώττιση). Η διαδικασία είναι απαραίτητη για να μετατρέψουμε αυτό που εμείς γράψαμε σε πηγαίο κώδικα (με τους συγκεκριμένους κανόνες από το λεξικό των λέξεων μιας γλώσσας προγραμματισμού) σε μια μορφή που να μπορεί ο Η/Υ να την καταλάβει και να την εκτελέσει.

Υπάρχουν δύο ειδών μεταφραστικά προγράμματα: οι **compilers** και οι **interpreters**.

Η διαφορά τους είναι κυρίως στον τρόπο με τον οποίο μεταφράζουν το πηγαίο πρόγραμμα σε γλώσσα μηχανής ΟΛΟΚΛΗΡΟ ή ΑΝΑ ΓΡΑΜΜΗ.

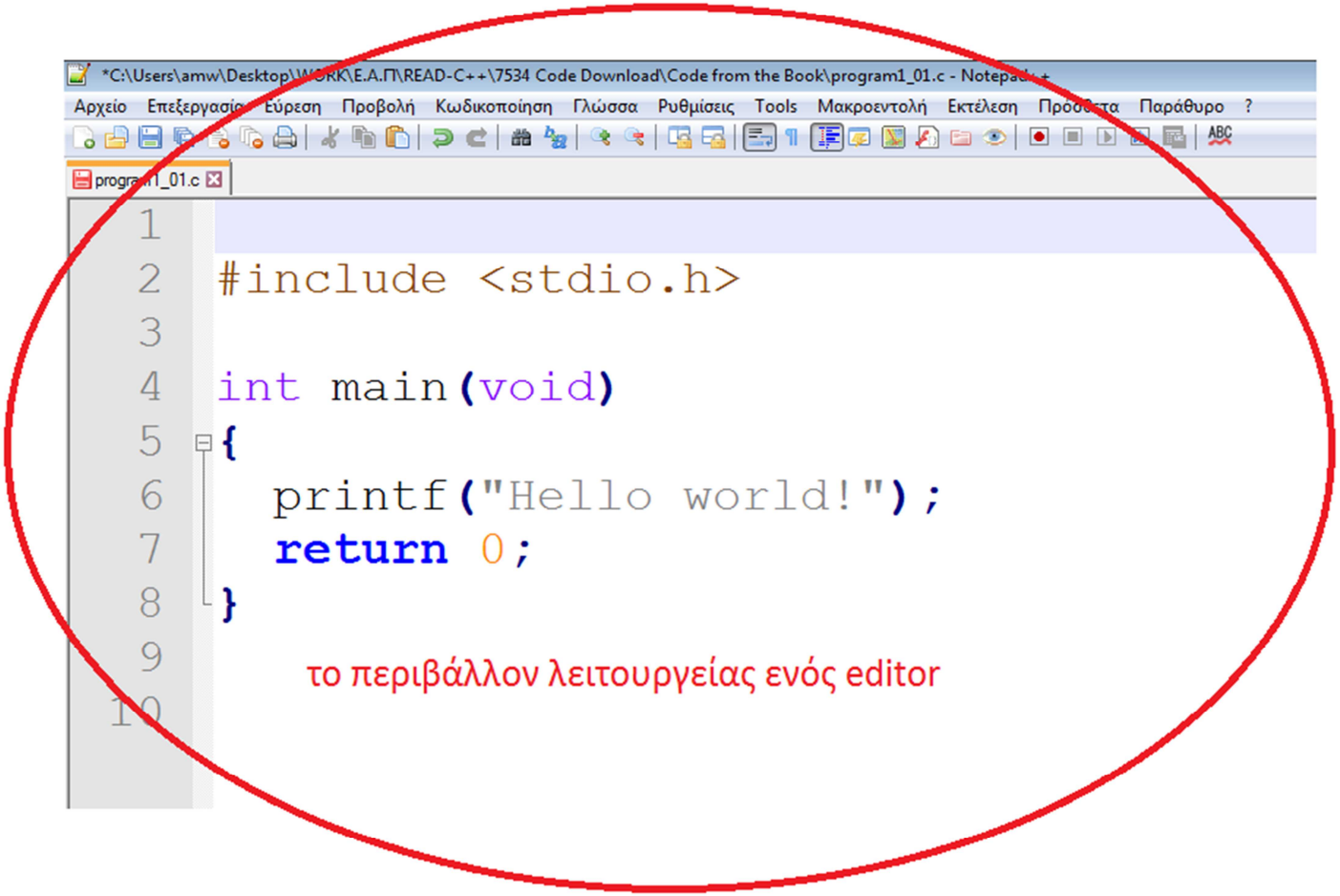
Οι **compilers** δημιουργούν ολόκληρο το εκτελέσιμο αντικείμενο (πρόγραμμα) το οποίο αν είναι απαλλαγμένο από λάθη δεν χρειάζεται να ξαναμεταφρασθεί όταν θα θελήσουμε να το ξανατρέξουμε. Αυτό δίνει μεγάλη ταχύτητα εκτέλεσης του προγράμματος.

Οι **interpreters** μεταφράζουν την κάθε γραμμή του προγράμματος ξεχωριστά, και εάν δεν ανιχνεύσουν λάθη την δίνουν στον Η/Υ για να την εκτελέσει χωρίς να την αποθηκεύουν κάπου μεταφρασμένη (σε εκτελέσιμη μορφή). Στην συνέχεια παίρνουν την επόμενη γραμμή του προγράμματος και επαναλαμβάνουν την διαδικασία της μετάφρασης για την νέα γραμμή, συνεχίζοντας μέχρι το τέλος του προγράμματος. Βασικό μειονέκτημα των interpreters είναι το ότι είναι πολύ αργοί.

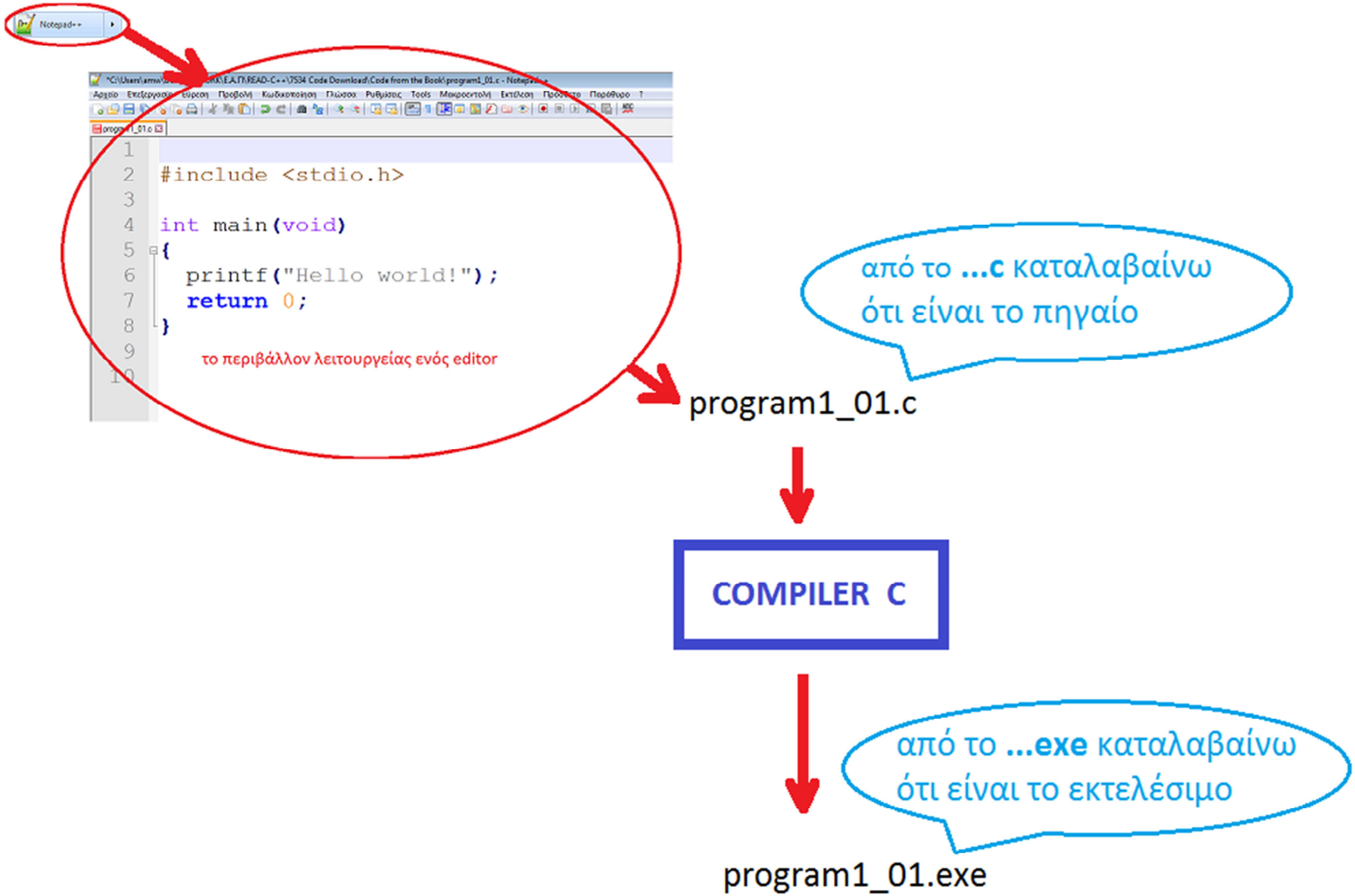
Μ.Ε. 2021-2022

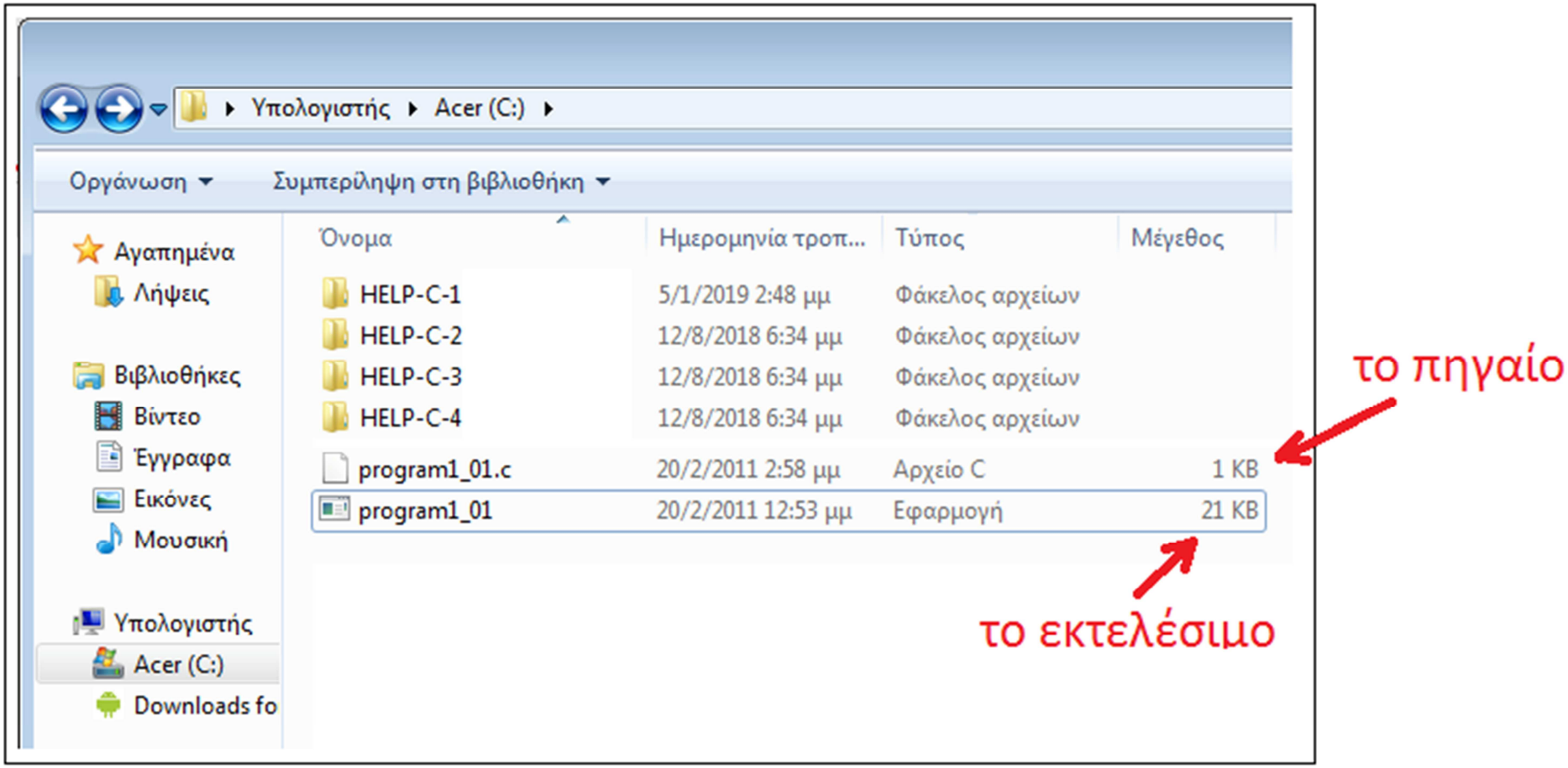
Για να δημιουργήσουμε ένα πρόγραμμα σε C, αρχικά πρέπει να γράψουμε τις εντολές σε μορφή κειμένου με κάποιον text editor. Υπάρχουν πολλοί text editors (με διαφορετικά χαρακτηριστικά ο καθένας) αλλά προτείνεται το NotePad++.

ο editor



το περιβάλλον λειτουργείας ενός editor


Notepad++
#include <stdio.h>
int main(void)
{
printf("Hello world!");
return 0;
}
το περιβάλλον λειτουργείας ενός editor
από το ...c καταλαβαίνω ότι είναι το πηγαίο
program1_01.c
COMPILER C
από το ...exe καταλαβαίνω ότι είναι το εκτελέσιμο
program1_01.exe


Υπολογιστής ▸ Acer (C:) ▸
Οργάνωση
Συμπερίληψη στη βιβλιοθήκη
Αγαπημένα
Λήψεις
Βιβλιοθήκες
Βίντεο
Έγγραφα
Εικόνες
Μουσική
Υπολογιστής
Acer (C:)
Downloads fo
Όνομα
Ημερομηνία τροπ...
Τύπος
Μέγεθος
HELP-C-1 5/1/2019 2:48 μμ Φάκελος αρχείων
HELP-C-2 12/8/2018 6:34 μμ Φάκελος αρχείων
HELP-C-3 12/8/2018 6:34 μμ Φάκελος αρχείων
HELP-C-4 12/8/2018 6:34 μμ Φάκελος αρχείων
program1_01.c 20/2/2011 2:58 μμ Αρχείο C 1 KB
program1_01 20/2/2011 12:53 μμ Εφαρμογή 21 KB
το πηγαίο
το εκτελέσιμο

Η ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C, C++

Η γλώσσα C είναι συνδεδεμένη με το λειτουργικό σύστημα UNIX. Δημιουργήθηκε στις αρχές της δεκαετίας του 1970 με σκοπό να είναι μια γλώσσα προγραμματισμού που θα διευκόλυνε την δημιουργία ενός νέου λειτουργικού συστήματος το οποίο θα ήταν γραμμένο σε μια εύκολα μεταφερόμενη γλώσσα υψηλού επιπέδου.

Ένα βασικό πλεονέκτημα της γλώσσας C, είναι ότι διαθέτει ΜΟΝΟ ένα πολύ μικρό κομμάτι της γραμμένο σε γλώσσα assembly (τη γλώσσα που βρίσκεται στο χαμηλότερο σχεδόν επίπεδο -πλησιέστερα στη μηχανή-).

Αυτό το χαρακτηριστικό βοήθησε το λειτουργικό σύστημα UNIX να λειτουργήσει σε όλων των ειδών τις μηχανές με σχετικά μεγάλη ευκολία, αφού το machine-dependent κομμάτι του ήταν περιορισμένο στο ελάχιστο δυνατό. Έτσι, όταν κάποιος ήθελε να μεταφέρει το UNIX σε έναν νέο Η/Υ, αρκούσε απλά να αλλάξει μόνο αυτό το μικρό κομμάτι του κώδικα του λειτουργικού συστήματος, που ήταν κοντά στην μηχανή. Για το υπόλοιπο αρκούσε να έχει ένα C compiler για τη συγκεκριμένη νέα μηχανή.

Η C, διαθέτει επίσης τον preprossesor (καλείται cpp στο Unix) μια μορφή προ-επεξεργασίας του προγράμματος, σε επίπεδο κειμένου, προτού γίνει η μεταγλώττιση στον compiler.

Στην δεκαετία του 1980 έκανε την εμφάνισή της και η C++, σαν μια επέκταση της C προς την κατεύθυνση του object oriented προγραμματισμού. Προσφέρεται ειδικά για δημιουργία φιλικών user interfaces (περιβάλλον αλληλεπίδρασης με τους χρήστες).

Στην δεκαετία του 2000 έκανε την εμφάνισή της και η C#, σαν μια επέκταση της C++ και σαν ένα αντίγραφο της Java. Η C#, κινείται επίσης προς την κατεύθυνση του object oriented προγραμματισμού.

Μ.Ι.Ζ. 2021-2022

ΤΟ ΑΛΦΑΒΗΤΟ ΣΤΙΣ ΓΛΩΣΣΕΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ

Το αλφάβητο της γλώσσας αποτελείται από τα γράμματα του ελληνικού και του λατινικού αλφαβήτου, τα αριθμητικά ψηφία, καθώς και κάποια ειδικά σύμβολα,

Γράμματα

Κεφαλαία ελληνικού αλφαβήτου (Α-Ω)
Πεζά ελληνικού αλφαβήτου (α-ω)
Κεφαλαία λατινικού αλφαβήτου (A-Z)
Πεζά λατινικού αλφαβήτου (a-z)

Ψηφία

0-9

Ειδικοί χαρακτήρες

+ - * / = ^ ( ) . , ‘ “ ! & και το κενό

Μ.Ι.Σ. 2021-2022

ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ ΣΤΙΣ ΓΛΩΣΣΕΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ

Οι υπολογιστές επεξεργάζονται δεδομένα διαφόρων τύπων, για αυτό είναι σημαντικό να κατανοήσουμε τους διαφορετικούς τύπους δεδομένων που χειρίζεται μια γλώσσα προγραμματισμού. Οι τύποι δεδομένων που υποστηρίζει μια γλώσσα προγραμματισμού είναι οι αριθμητικοί, που περιλαμβάνουν τους ακέραιους και τους πραγματικούς αριθμούς, οι χαρακτήρες και οι λογικοί τύποι.

Ακέραιος τύπος. Ο τύπος αυτός περιλαμβάνει τους ακέραιους που είναι γνωστοί από τα μαθηματικά. Οι ακέραιοι μπορούν να είναι θετικοί, αρνητικοί ή μηδέν (π.χ. 1, 67419, 0, -12980.

Πραγματικός τύπος. Ο τύπος αυτός περιλαμβάνει τους πραγματικούς αριθμούς που γνωρίζουμε από τα μαθηματικά (π.χ. 33.156159, 22.743528, -452.45, 0.445). Οι πραγματικοί αριθμοί μπορούν να είναι θετικοί, αρνητικοί ή μηδέν.

Χαρακτήρες (αλφαριθμητικά). Ο τύπος αυτός αναφέρεται τόσο σε ένα χαρακτήρα όσο και μία σειρά χαρακτήρων. Τα δεδομένα αυτού του τύπου μπορούν να περιέχουν οποιοδήποτε χαρακτήρα παράγεται από το πληκτρολόγιο (π.χ. ‘Κ’, ‘Κώστας’, ‘σήμερα είναι καλά, ‘Τα πρόβατα είναι 2’, ‘είναι το 3.14)’. Οι χαρακτήρες πρέπει υποχρεωτικά να βρίσκονται μέσα σε εισαγωγικά, ‘ ’.

Λογικός. τύπος που δέχεται μόνο δύο τιμές ΑΛΗΘΗΣ και ΨΕΥΔΗΣ. Οι τιμές αντιπροσωπεύουν αληθείς ή ψευδείς συνθήκες.

Μ.Ι.Σ. 2021-2022

ΣΤΑΘΕΡΕΣ

Οι σταθερές (constants) είναι προκαθορισμένες τιμές που δεν μεταβάλλονται κατά τη διάρκεια εκτέλεσης ενός προγράμματος. Οι σταθερές είναι αντίστοιχου τύπου δεδομένων, δηλαδή ακέραιες, πραγματικές, αλφαριθμητικές ή λογικές.

ΜΕΤΑΒΛΗΤΕΣ

Η έννοια της μεταβλητής (variable) είναι γνωστή από τα μαθηματικά (π.χ. στον τύπο **Ε=α*β**). Μια μεταβλητή, παριστάνει μία ποσότητα που η τιμή της μπορεί να μεταβάλλεται. Οι μεταβλητές που χρησιμοποιούνται σε ένα πρόγραμμα, αντιστοιχούνται από τον compiler σε συγκεκριμένες θέσεις μνήμης του υπολογιστή. Η τιμή της μεταβλητής είναι η τιμή που βρίσκεται στην αντίστοιχη θέση μνήμης και όπως αναφέρθηκε μπορεί να μεταβάλλεται κατά τη διάρκεια της εκτέλεσης του προγράμματος. Μπορούμε να παρομοιάσουμε τη μεταβλητή και την αντίστοιχη θέση μνήμης σαν ένα κουτί, το οποίο εξωτερικά έχει ως όνομα το όνομα της μεταβλητής και ως περιεχόμενο εσωτερικά, την τιμή που έχει εκείνη τη συγκεκριμένη στιγμή η μεταβλητή. Ενώ η τιμή της μεταβλητής μπορεί να αλλάζει κατά την εκτέλεση του προγράμματος, αυτό που μένει υποχρεωτικά αναλλοίωτο είναι ο τύπος και το όνομα της μεταβλητής. Η **ΓΛΩΣΣΑ** επιτρέπει τη χρήση μεταβλητών από τους 4 τύπους (ακεραίων, πραγματικών, χαρακτήρων και λογικών).

ΟΝΟΜΑΣΙΕΣ ΜΕΤΑΒΛΗΤΩΝ & ΣΤΑΘΕΡΩΝ

Κάθε πρόγραμμα, καθώς και τα δεδομένα που χρησιμοποιεί έχουν ένα όνομα, με το οποίο αναφερόμαστε σε αυτά. Τα ονόματα αυτά μπορούν να αποτελούνται από γράμματα πεζά ή κεφαλαία του λατινικού αλφαβήτου, ψηφία (0-9) καθώς και τον χαρακτήρα κάτω παύλα (underscore) (_) ή την κανονική παύλα (-), ενώ πρέπει υποχρεωτικά να αρχίζουν με γράμμα (π.χ. A-10, MyValue, MY-Value, A-this).

Η C κάνει διάκριση μεταξύ κεφαλαίων και μικρών γραμμάτων. Για παράδειγμα, τα **nikos** και **NIKOS** είναι δύο διαφορετικά αναγνωριστικά. Ένα αναγνωριστικό δεν μπορεί να είναι το ίδιο με κάποια λέξη κλειδί της C, και δεν πρέπει να έχει το ίδιο όνομα με τις συναρτήσεις που έχουμε γράψει είτε υπάρχουν στην βιβλιοθήκη της C.

Μ.Π.Σ. 2021-2022

ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ ΣΤΗΝ C

Όλες οι μεταβλητές στην C πρέπει να δηλώνονται πριν χρησιμοποιηθούν. Αυτό χρειάζεται γιατί η γλώσσα πρέπει να γνωρίζει τον τύπο δεδομένων μιας μεταβλητής για να μπορέσει να μεταγλωττίσει σωστά τις εντολές που περιέχουν τη συγκεκριμένη μεταβλητή.

Στην γλώσσα C, υπάρχουν τέσσερις βασικοί τύποι δεδομένων:

- **χαρακτήρας**
- **ακέραιος**
- **κινητής υποδιαστολής**
- **κινητής υποδιαστολής χωρίς ακρίβεια.**

Και οι «εργοστασιακές λέξεις» που χρησιμοποιούνται για να δηλώσουν τις μεταβλητές αυτών των τύπων είναι οι,

- **char**
- **int**
- **float**
- **double**

Ειδικότερα

- οι μεταβλητές τύπου **char** χρησιμοποιούνται για να κρατάνε χαρακτήρες ASCII σαν τους **l**, **d**, **f**
  ΠΡΟΣΟΧΗ εσωτερικά η κλασική C τους χειρίζεται σαν αριθμούς
- οι μεταβλητές τύπου **int** μπορούν να περιέχουν ακέραιες ποσότητες χωρίς κλασματικό μέρος
- οι μεταβλητές τύπου **float** χρησιμοποιούνται όταν απαιτούνται κλασματικά μέρη
- οι μεταβλητές τύπου **double** χρησιμοποιούνται όταν απαιτούνται κλασματικά μέρη και αποτύπωση πολύ μεγάλων αριθμών

Μ.Ι.Σ. 2021-2022

Τύπος	πλάτος σε bit	Εύρος
char	8	0 έως 255
int	16	-32768 έως 32767
float	32	3.4E -38 έως 3.4E + 38
double	64	1.7E – 308 έως 1.7E + 308

Η C δεν έχει λογικό τύπο, κάποιο τύπο δηλαδή του οποίου οι τιμές να είναι είτε true (αληθής) είτε false (ψευδής).

Μ.Ι.Σ. 2021

ΤΡΟΠΟΠΟΙΗΤΕΣ ΤΥΠΩΝ ΔΕΔΟΜΕΝΩΝ ΣΤΗΝ C

Οι βασικοί τύποι δεδομένων στην C μπορούν να έχουν μπροστά τους διάφορους τροποποιητές (modifiers).

Οι τροποποιητές χρησιμοποιούνται για να αλλάζουν την σημασία του βασικού τύπου ώστε να ικανοποιεί καλύτερα της ανάγκες κάποιου προγράμματος.

- **signed -** προσημασμένος
- **unsigned -** χωρίς πρόσημο
- **long -** μεγάλος
- **short -** μικρός

Μπορούμε να χρησιμοποιούμε τους τροποποιητές **signed**, **unsigned**, **long** και **short** μπροστά από τους βασικούς τύπους **char** και **int**. Μπορούμε πάντως να χρησιμοποιούμε και το **long** μπροστά από το **double**. Παρ' όλο που επιτρέπεται, η χρήση του **signed** στους ακέραιους, αυτό είναι περιττό, γιατί η από ορισμού δήλωση του ακεραίου αφορά έναν προσημασμένο αριθμό.

Μ.Ι.Σ. 2021-2022

Τύπος	πλάτος σε bit	Εύρος
char	8	-128 έως 127
unsigned char	8	0 έως 255
signed char	8	-128 έως 127
int	16	-32768 έως 32767
unsigned int	16	0 έως 65535
signed int	16	-32768 έως 32767
short int	16	-32768 έως 32767
unsigned short int	16	0 έως 65535
signed short int	16	-32768 έως 32767
long int	32	-2147483648 έως 2147483649
signed long int	32	-2147483648 έως 2147483649
unsigned long int	32	0 έως 4294967296
float	32	3.4^{E} -38 έως 3.4E +38
double	64	1.7^{E} – 308 έως 1.7E + 308
long double	64	3.4^{E} – 4932 έως 1.1E + 4932

ΠΩΣ ΔΗΛΩΝΟΥΜΕ ΜΙΑ ΜΕΤΑΒΛΗΤΗ ΣΤΗΝ C

Η γενική μορφή δήλωσης μιας μεταβλητής στην C, είναι :

Τύπος μεταβλητής λίστα_μεταβλητών;

Παράδειγμα:

```
#include <stdio.h>
int main()
{
        int a, b, c, x;
        float MyBalance;
```

Στην C, μπορούμε να δίνουμε αρχικές τιμές στις μεταβλητές την στιγμή που τις δηλώνουμε.

τύπος όνομα_μεταβλητής = σταθερά;

Παράδειγμα:

```
#include <stdio.h>
int main()
{
        int m = 0;
        int t   = 82;
```

Μ.Ι.Σ. 2021-2022

ΠΩΣ ΔΗΛΩΝΟΥΜΕ ΣΤΗΝ C, ΕΙΔΙΚΕΣ ΣΤΑΘΕΡΕΣ ΜΕ ΤΗΝ ΔΗΛΩΣΗ #define

Οι γραμμές ενός προγράμματος που βρίσκονται στην αρχή και αρχίζουν με τη «εργοστασιακή» λέξη #define αποτελούν εντολές για τον cpp (pre-compiler) και όχι για τον ίδιο τον compiler.

Το νόημα της εντολής #define *name text* είναι, ότι ζητάμε από τον cpp οπουδήποτε βλέπει το *name* μέσα στο κείμενο του προγράμματος μας να το αντικαθιστά με το κείμενο *text*.

Το *name* πρέπει να είναι του ίδιου τύπου με αυτά που χρησιμοποιούνται για μεταβλητές, δηλαδή να αρχίζει με γράμμα ή underscore (_) και να περιέχει μόνο γράμματα, αριθμούς και underscores, αλλά το *text* είναι τελείως ελεύθερο στον προγραμματιστή.

Παράδειγμα:

```
#define  ISOTIMIA  340.74
```

ο cpp, θα αντικαταστήσει μέσα στο πρόγραμμα οποιαδήποτε εμφάνιση του ISOTIMIA με το 340.74

Γιατί το κάνουμε αυτό και δε γράφουμε κατ' ευθείαν το 340.74 εκεί που το χρειαζόμαστε; Η απάντηση είναι απλή: γιατί το ISOTIMIA παριστάνει μια δήλωση που μπορεί να χρησιμοποιείται σε πάρα πολλά σημεία στο πρόγραμμα και άρα να είναι δυσκολότερη η αλλαγή εάν αλλάξει π.χ. το 340,74 σε 130.80.

Παράδειγμα:

```
#define DEFAULT_PRICE 29
int main()
{
        my_val = DEFAULT_PRICE ;
```

M.I.S. 2021-2022

ΣΤΑΘΕΡΕΣ ΠΟΥ ΞΕΚΙΝΟΥΝ ΜΕ ΤΟΝ ΧΑΡΑΚΤΗΡΑ \

Για να δηλώσουμε σε ένα αλφαριθμητικό πεδίο (μεταβλητή) ένα σύνολο χαρακτήρων το κάνουμε με την χρήση διπλών εισαγωγικών. Για παράδειγμα, το «**είναι το πρώτο μου πρόγραμμα**» είναι ένα γέμισμα ενός αλφαριθμητικού πεδίου με το απλό αυτό κείμενο.

Εάν όμως θέλουμε, για παράδειγμα να αλλάξουμε γραμμή στον εκτυπωτή μας, η C παρέχει τις παρακάτω <u>ειδικές σταθερές ανάποδης καθέτου</u>,

Κδ	Σημασία
\b	οπισθοχώρηση
\f	αλλαγή σελίδας
\n	νέα γραμμή
\r	επαναφορά κεφαλής
\t	οριζόντιος στηλογνώμονας
\”	διπλό εισαγωγικό
\’	απλό εισαγωγικό
\?	λατινικό ερωτηματικό
****	ανάποδη κάθετος
\v	κατακόρυφος στηλογνώμονας
\000	οκταδικός αριθμός
\xhhh	δεκαεξαδικός αριθμός

Μ.Ι.Ζ. 2021-2022

ΟΙ ΤΕΛΕΣΤΕΣ ΣΤΗΝ C

Οι τελεστές (operators) είναι τα σύμβολα προς τον μεταγλωττιστή compiler να εκτελέσει μαθηματικές ή λογικές πράξεις.

Η C έχει τρείς γενικές κατηγορίες τελεστών:

- αριθμητικούς (arithmetic)
- συσχετιστικούς (relational) και λογικούς (logical)
- τελεστές για bit (bitwise)

ΑΡΙΘΜΗΤΙΚΟΙ ΤΕΛΕΣΤΕΣ

- \- αφαίρεση
- \- αρνητικό πρόσημο (αν τοποθετηθεί μπροστά από αριθμό, θα κάνει αλλαγή του πρόσημου του)
- \+ πρόσθεση
- * πολλαπλασιασμός
- / διαίρεση
- % ακέραιο υπόλοιπο διαίρεσης (πράξη modulo, επιστρέφει το υπόλοιπο της διαίρεσης)
- -- μείωση κατά ένα
- ++ αύξηση κατά ένα

<u>Σημείωση</u>, Η διαίρεση δύο ακεραίων θα δώσει ακέραιο αποτέλεσμα ενώ η διαίρεση πραγματικών αριθμών θα δώσει πραγματικό αποτέλεσμα.

Μ.Π.Σ. 2021-2022

Ο τελεστής % παράγει το υπόλοιπο μιας διαίρεσης ακεραίων και δεν μπορεί να χρησιμοποιηθεί με τους τύπους float ή double.

Παράδειγμα:

```
#include <stdio.h>
int main()
{
        int x, y;
        x = 10;
        y = 3;
        printf("%d\n",   x/y);                    // θα δώσει 3
        printf("%d\n",   x % y);                  // θα δώσει 1

        x = 1;
        y = 2;
        printf("%d %d\n",   x /y,   x % y);       // θα δώσει 0 και 1, γιατί το ½ στην διαίρεση ακεραίων είναι 0 με υπόλοιπο 1

        return 0;
}
```

Μ.Ι.Σ. 2021-2022

ΤΕΛΕΣΤΗΣ ΕΚΧΩΡΗΣΗΣ/ΑΝΑΘΕΣΗΣ ΤΙΜΗΣ

Ο τελεστής εκχώρησης τιμής στην C είναι το σύμβολο =

Ο τελεστής = εκχωρεί την τιμή από την δεξιά πλευρά στην μεταβλητή που είναι στην αριστερή πλευρά.

Παράδειγμα:

```
#include <stdio.h>
int main()
{
        int a;
        int b;
        int sum;

        printf("Δώστε μου δύο αριθμούς : ");
        scanf("%d %d",&a,&b);

        sum = a + b;

        printf("Το άθροισμα των δύο αριθμών είναι %d \n",sum);
        return 0;
}
```

Μ.Ι.Σ. 2021-2022

ΟΙ ΕΙΔΙΚΟΙ ΤΕΛΕΣΤΕΣ ΑΥΞΗΣΗΣ ΚΑΙ ΜΕΙΩΣΗΣ (ΚΑΤΑ ΕΝΑ)

Η C διαθέτει δυο χρήσιμους τελεστές. Τον τελεστή μοναδιαίας αύξησης ++ και τον τελεστή μοναδιαίας μείωσης --.

Ο τελεστής μοναδιαίας αύξησης προσθέτει 1 στον τελεστέο και ο τελεστής μοναδιαίας μείωσης αφαιρεί 1 από τον τελεστέο.

οι πράξεις:

x = x + 1;
x = x – 1;

είναι ίδιες με τις

x++;
x--;
++x;
--x;

Παραστάσεις όπως η

x = x + 4

όπου η αρχική μεταβλητή του αριστερού μέλους επαναλαμβάνεται αμέσως στην αρχή του δεξιού μέλους, μπορούν να γραφούν σαν

x += 4

Ο τελεστής **+=** λέγεται τελεστής αντικατάστασης (assignment operator).

Μ.Ι.Σ. 2021-2022

Οι τελεστές ++ και - - μπορούν να μπουν και μπροστά από μια μεταβλητή,

++x;

--x;

με την διαφορά ότι όταν ο τελεστής, βρίσκεται μπροστά από την μεταβλητή, η C εκτελεί την πράξη της αύξησης ή της μείωσης πριν χρησιμοποιήσει την τιμή της μεταβλητής. Ενώ όταν ο τελεστής βρίσκεται μετά την μεταβλητή, η C χρησιμοποιεί την τιμή της μεταβλητής πριν την αυξήσει ή την μειώσει.

Όταν γίνεται χρήση των τελεστών ++ και - - ταυτόχρονα με ανάθεση, θα πρέπει να λαμβάνεται υπόψη η προτεραιότητα των ενεργειών που εκτελεί η C

Παράδειγμα :
στον κώδικα

x = 12 ;

sum = ++x ;

η C εκτελεί την πράξη της αύξησης κατά 1, πριν χρησιμοποιήσει την τιμή της μεταβλητής.

η C, <u>δίνει στην **sum** την τιμή **13**</u> γιατί αυξάνει πρώτα την **x** και στη συνέχεια δίνει την τιμή του στην **sum**

αντίθετα στον κώδικα,

x = 12 ;

sum = x++ ;

η C, χρησιμοποιεί την τιμή της μεταβλητής πριν την πράξη της αύξησης κατά 1

η C, <u>θα είχε δώσει στην **sum** την τιμή **12**</u> και στη συνέχεια θα είχε αυξήσει κατά **1** την τιμή της **x**

Μ.Ι.Σ. 2021-2022

ΠΡΟΤΕΡΑΙΟΤΗΤΕΣ ΣΤΟΥΣ ΤΕΛΕΣΤΕΣ ΣΕ ΑΡΙΘΜΗΤΙΚΗ ΕΚΦΡΑΣΗ

Στην C, η προτεραιότητα των συμβόλων σε μια αριθμητική έκφραση είναι συγκεκριμένη,

ΥΨΗΛΟΤΕΡΗ

( )

++ --

*** / %**

+ -

ΧΑΜΗΛΟΤΕΡΗ

Όταν σε μια έκφραση εμφανίζονται τελεστές με ίδια προτεραιότητα, τότε ισχύει το ΑΠΟ-ΑΡΙΣΤΕΡΑ-ΠΡΟΣ-ΤΑ-ΔΕΞΙΑ

Με την χρήση των παρενθέσεων μπορούμε να αλλάξουμε τις προτεραιότητες

Μ.Ι.Σ. 2021-2022

Η ΔΗΛΩΣΗ #include ΚΑΙ Η ΕΝΣΩΜΑΤΩΣΗ «ΑΡΧΕΙΩΝ ΕΠΙΚΕΦΑΛΙΔΩΝ»

Στην C, υπάρχει μια ειδική δήλωση η οποία βρίσκεται στην αρχή κάθε C προγράμματος και απευθύνεται στον cpp, (τον C preprossesor), ζητώντας του να τοποθετήσει τα περιεχόμενα κάποιου εξωτερικού βοηθητικού αρχείου μέσα στο πρόγραμμα.

Για παράδειγμα η εντολή #include <stdio.h>

Προσθέτει το αρχείο stdio.h στη θέση της γραμμής #include.

Το αρχείο stdio.h (standard input/output header file) δεν περιέχει κάποιο εκτελέσιμο κώδικα, αλλά κάποιες δηλώσεις με σταθερές και υποπρογράμματα.

Αυτή η δήλωση γίνεται για κάθε αρχείο επικεφαλίδων που είναι απαραίτητο για την επιτυχή μετάφραση και εκτέλεση του κάθε προγράμματος.

Δηλώνουμε δηλαδή, ποιες από τις βιβλιοθήκες θα ενσωματωθούν με το κυρίως πρόγραμμα (εάν είναι απαραίτητες).

Αυτό γίνεται με την οδηγία προς τον προεπεξεργαστή της C με την εντολή:

#include <όνομα βιβλιοθήκης.h>

Πώς γνωρίζαμε ποιό/ποιά αρχείο να κάνουμε #include;

Απάντηση… βρίσκεται στο manual της γλώσσας C που χρησιμοποιούμε.

Μ.Ι.Σ. 2021-2022

Αρχείο	**Περιγραφή Συναρτήσεων**
stdarg.h	Διαχείρισης Μεταβλητών Λιστών Ορισμάτων
io.h	Διαχείρισης εισόδου / εξόδου σε χαμηλό επίπεδο
limits.h & float.h	Ορίζει τα όρια των ακεραίων και των αριθμών κινητής υποδιαστολής.
math.h	Μαθηματικές Συναρτήσεις
mem.h	Διαχείριση Δυναμικής Μνήμης
stdio.h	Διαχείριση εισόδου και εξόδου σε υψηλό επίπεδο
stdlib.h	Συχνά χρησιμοποιούμενες συναρτήσεις
string.h & ctype.h	Συναρτήσεις Επεξεργασίας αλφαριθμητικών και χαρακτήρων
time.h	Διαχείρισης ώρας και ημερομηνίας και επεξεργασίας χρόνου

Η βιβλιοθήκη **stdio.h** (STandard Input Output) παρέχει το πρόγραμμα με βοηθητικές συναρτήσεις που συσχετίζονται με τις ροές εισόδου και εξόδου. Η ροή εισόδου σχετίζεται με τι θα χρειαστεί ένα πρόγραμμα από τον έξω κόσμο (π.χ. πληκτρολόγιο) και η ροή εξόδου με το τι θα δείξει ένα πρόγραμμα στον έξω κόσμο (π.χ. τι θα εκτυπώσει και πού).
Η προέκταση **.h** υποδηλώνει ότι πρόκειται για αρχείο βιβλιοθήκη (header file).

ΣΧΟΛΙΑ-COMMENTS ΣΤΗΝ C

Στην C, μπορούμε να τοποθετούμε σχόλια οπουδήποτε μέσα στο πρόγραμμα. Τα σχόλια βρίσκονται ανάμεσα σε δύο ειδικούς χαρακτήρες. Ο ειδικός χαρακτήρας της αρχής του σχολίου είναι ο **/*** ενώ ο ειδικός χαρακτήρας του τέλους του σχολίου είναι το ***/**

Αυτά τα *σχόλια* σκοπό έχουν να δώσουν τη δυνατότητα στον προγραμματιστή να συμπεριλαμβάνει κάποιες επεξηγήσεις

```
/*
αυτό είναι ένα σχόλιο 1η γραμμή
αυτό είναι ένα σχόλιο 2η γραμμή
αυτό είναι ένα σχόλιο 3η γραμμή
*/
```

Αν θέλουμε να χρησιμοποιήσουμε σχόλια σε μια μόνο γραμμή γράφουμε

```
// αυτό είναι ένα σχόλιο μιας γραμμής
```

Μ.Ι.Σ. 2021-2022

Ο ΒΑΣΙΚΟΣ ΚΟΡΜΟΣ ΕΝΟΣ ΠΡΟΓΡΑΜΜΑΤΟΣ C

Κάθε πρόγραμμα C, πρέπει να περιέχει μία συνάρτηση που ονομάζεται **main( )** και είναι το σημείο που αρχίζει ο κώδικας.

Οι παρενθέσεις που ακολουθούν την main υποδηλώνουν ότι αυτή η συνάρτηση δεν έχει παραμέτρους (θα μπορούσε να δέχεται παραμέτρους). Η λέξη **int** στην αρχή δηλώνει ότι η συνάρτηση **main( )** θα επιστρέψει έναν ακέραιο αριθμό (αυτό που στα μαθηματικά ονομάζεται πεδίο τιμών συνάρτησης ενώ μέσα στις παρενθέσεις δίνεται το πεδίο ορισμού).

Τα άγκιστρα **{ }** δείχνουν που αρχίζει και που τελειώνει το πρόγραμμα.

`#include <stdio.h>`
`int main()`
`{`
`  printf("Hello World 1 !\n");`
`  return 0;`
`}`
	`#include <stdio.h>`
	`main()`
	`{`
	`  printf("Hello World 2 !\n");`
	`}`

Η **printf( )** είναι μία εντολή εκτύπωσης με δυνατότητες μορφοποίησης. Εμπεριέχεται στην βιβλιοθήκη **stdio.h**. Θα εκτυπώσει ότι υπάρχει μέσα στα εισαγωγικά εκτός από τους χαρακτήρες που ακολουθούν την ανάποδη κάθετο, για παράδειγμα η \n ζητά από την **printf( )** να πάει σε επόμενη νέα γραμμή.
Η εντολή **return** δηλώνει την επιστροφή του προγράμματος, όπου στην περίπτωση μας, δεν γυρίζει τίποτα, 0 (αυτή η εντολή εάν δεν δηλωθεί η main σαν int δεν χρειάζεται να γραφεί).

Μ.Ι.Σ. 2021-2022

ΟΙ ΕΡΓΟΣΤΑΣΙΑΚΕΣ ΛΕΞΕΙΣ ΣΤΗΝ C

Η C, αποτελείται από λέξεις–κλειδιά-keywords «ΕΡΓΟΣΤΑΣΙΑΚΕΣ» και από συντακτικούς κανόνες οι οποίοι εφαρμόζονται επάνω σε κάθε λέξη–κλειδί-keyword. Μια λέξη–κλειδί-keyword.

Λέξεις–κλειδιά-keywords (κατά το πρότυπο ANSI),

auto	**double**	**int**	**struct**
break	**else**	**long**	**switch**
case	**enum**	**register**	**typedef**
char	**extern**	**return**	**union**
const	**float**	**short**	**unsigned**
continue	**for**	**signed**	**void**
default	**goto**	**sizeof**	**volatile**
do	**if**	**static**	**while**

- όλες οι λέξεις–κλειδιά-keywords της C είναι με μικρά γράμματα
- η C κάνει διάκριση κεφαλαίων – μικρών, έτσι το **while** είναι μια λέξη–κλειδί-keyword, ενώ το **WHILE** δεν είναι
- φυσικά, δεν επιτρέπετε να χρησιμοποιηθεί μια λέξη–κλειδί-keyword σαν όνομα μεταβλητής

Η ΕΝΤΟΛΗ printf()

Με την **printf()** η C, προβάλει στην οθόνη και η σύνταξη της είναι

printf("μορφοποίηση…", λίστα ορισμάτων)

Το *"μορφοποίηση…",* περιέχει

1) απλούς χαρακτήρες που θα εμφανιστούν στην οθόνη
2) <u>ειδικούς χαρακτήρες μορφοποίησης</u> που λένε στην **printf()** πώς να εμφανίσει το περιεχόμενο από τα ορίσματα
3) και απλούς χαρακτήρες και <u>ειδικούς χαρακτήρες μορφοποίησης</u>

Μια εντολή μορφοποίησης, περιέχει πρώτα το σύμβολο % και στην συνέχεια τον <u>ειδικό χαρακτήρα μορφοποίησης</u>. Ο αριθμός των ορισμάτων πρέπει να είναι ίσος με τον αριθμό των <u>ειδικών χαρακτήρων μορφοποίησης</u> και φυσικά να έχουν την ίδια σειρά.

Για παράδειγμα, η εντολή

```
printf( "%d  %c  %s",  5,  'a',  "ΔΕΛΤΑ" );
```

Θα εμφανίσει στην οθόνη το,

```
5  a  ΔΕΛΤΑ
```

Αντίστοιχα τα παράδειγμα

```
printf ( "%-5.2f",   823.234 ) ;        823.23
printf ( "%5.2.f",   4.234 ) ;               4.23
printf ( "%10s",   "trello" );          _____trello
printf ( "%-10s",   "trello" );         trello_____
```

Μ.Ι.Σ. 2021-2022

Οι <u>ειδικοί χαρακτήρες μορφοποίησης</u> είναι οι,

Προσδιοριστής	Περιγραφή
d,i	Προσημασμένος ακέραιος αριθμός
u	Μη Προσημασμένος ακέραιος αριθμός
c	απλός χαρακτήρας
x	Μη προσημασμένος ακέραιος σε δεκαεξαδική μορφή όπου τα a, b, c, d, e, f τυπώνονται πεζά
X	Μη προσημασμένος ακέραιος σε δεκαεξαδική μορφή όπου τα a, b, c, d, e, f τυπώνονται κεφαλαία
o	Μη προσημασμένος ακέραιος σε οκταδική μορφή
f	Αριθμός κινητής υποδιαστολής της μορφής $-m.dddddd$ όπου το πλήθος των d καθορίζεται από την ακρίβεια η οποία εξ' ορισμού είναι 6 και μπορεί να αλλάξει από τον προγραμματιστή στη φόρμα μετατροπής
e, E	Αριθμός κινητής υποδιαστολής της μορφής $-m.dddddde+/-xx$ ή $-m.ddddddE+/-xx$ όπου το πλήθος των d καθορίζεται από την ακρίβεια η οποία εξ' ορισμού είναι 6 και μπορεί να αλλάξει από τον προγραμματιστή στο φόρμα μετατροπής
g, G	όταν ο εκθέτης είναι μικρότερος του −4 ή μεγαλύτερος ή ίσος από την ακρίβεια τότε χρησιμοποιείται το %e/%E διαφορετικά το %f
s	Αλφαριθμητικό
p	το ορισμά που αντιστοιχεί στον προσδιοριστή αυτό τυπώνεται ως δείκτης

Η ΕΝΤΟΛΗ scanf()

Με την **scanf()** η C, διαβάζει από το πληκτρολόγιο και γεμίζει μεταβλητές. Η σύνταξη της είναι ,

scanf("μορφοποίηση…", λίστα ορισμάτων)

Το *"μορφοποίηση…",* περιέχει τους <u>ειδικούς χαρακτήρες μορφοποίησης</u> που λένε στην **scanf ()** πώς να εμφανίσει τα ορίσματα.

Μια εντολή μορφοποίησης, περιέχει πρώτα το σύμβολο % και στην συνέχεια τον <u>ειδικό χαρακτήρα μορφοποίησης</u>. Ο αριθμός των ορισμάτων πρέπει να είναι ίσος με τον αριθμό των <u>ειδικών χαρακτήρων μορφοποίησης</u> και φυσικά να έχουν την ίδια σειρά.

Η **scanf()** παίρνει πάντα ως πρώτη παράμετρο το *"μορφοποίηση…"* το οποίο το φτιάχνουμε έτσι ώστε να δηλώνουμε στην **scanf()** τι τύπους δεδομένων θα διαβάσουμε.

Οι επιπλέον παράμετροι (*λίστα ορισμάτων)* θα είναι τόσες, όσα και τα % που βρίσκονται μέσα στο *"μορφοποίηση…"* και θα παριστάνουν η καθεμία από μια θέση μνήμης (μεταβλητές) που έχουμε να διαβάσουμε και να γεμίσουμε.

Μ.Ι.Σ. 2021-2022

Με τις γραμμές,

```
int   my_integer;
scanf("%d", &my_integer);
```

η C, διαβάζει ένα ακέραιο από το πληκτρολόγιο και τον αποθηκεύει στη μεταβλητή my_integer που δηλώσαμε αρχικά.

Μέσα στο *"μορφοποίηση…"* δηλώνουμε τι είναι αυτό που θα διαβάσει η C από το πληκτρολόγιο.
Στην περίπτωση μας, δηλώνουμε ότι θα διαβάσουμε έναν προσημασμένο δεκαδικό ακέραιο (για αυτό ορίζουμε το d που είναι το decimal)
Και θέλουμε η C να τοποθετήσει αυτό που θα διαβάσει στη θέση my_integer

Στη C, για να αναφερθούμε σε αυτή τη θέση μνήμης για την μεταβλητή με όνομα my_integer γράφουμε απλά &my_integer.

αυτή η θέση μνήμης, λέγεται και διεύθυνση (*address*) του my_integer.

Αντίστοιχα για να διαβάσουμε τους αριθμούς a, b και x θα γράψουμε;

```
Int          my_a, my_b;
double       my_x;
scanf( "%d %d %lf ",  & my_a,  & my_b,  & my_x);
```

Μ.Ι.Σ. 2021-2022

Ειδικότερα το *"μορφοποίηση…",* μπορεί να περιέχει, τρεις κατηγορίες χαρακτήρων :

- καθοριστικά φόρμας (<u>ειδικό χαρακτήρα μορφοποίησης)</u>
- χαρακτήρες κενών
- χαρακτήρες μη-κενών

Μπροστά από τα καθοριστικά φόρμας τοποθετείται ένα σύμβολο %.

Ένας χαρακτήρας κενού είναι ένα κενό διάστημα ή ένα tab ή μια νέα γραμμή. Ουσιαστικά, ένας χαρακτήρας κενού μέσα στο αλφαριθμητικό ελέγχου υποχρεώνει την **scanf()** να διαβάσει, αλλά όχι και να αποθηκεύσει, οποιονδήποτε αριθμό (συμπεριλαμβανομένου του μηδενός) κενών χαρακτήρων, μέχρι να συναντήσει τον πρώτο χαρακτήρας μη-κενού.

Ένας χαρακτήρας μη-κενού μέσα στο *"μορφοποίηση…",* υποχρεώνει τη **scanf()** να διαβάζει και να απορρίπτει κάποιο συγκεκριμένο χαρακτήρα. Για παράδειγμα, το *"μορφοποίηση…",* **"%d,%d"** υποχρεώνει την **scanf()** πρώτα να διαβάσει έναν ακέραιο, στη συνέχεια να διαβάζει και να απορρίπτει ένα κόμμα και τέλος να διαβάζει άλλον έναν ακέραιο. Αν ο υπολογιστής δεν βρεί τον χαρακτήρα που έχει καθοριστεί, η **scanf()** θα τερματιστεί.

Όλες οι μεταβλητές που χρησιμοποιούνται για να δέχονται τιμές μέσω της **scanf()** πρέπει να περνάνε μέσω των διευθύνσεων τους. Αυτό σημαίνει ότι όλα τα ορίσματα πρέπει να είναι δείκτες στις μεταβλητές που χρησιμοποιούνται σαν ορίσματα. Για παράδειγμα, για να διαβαστεί ένας ακέραιος στη μεταβλητή **count**, θα χρησιμοποιήσουμε την παρακάτω κλήση της **scanf()** scanf("%d",&count);

Η συνάρτηση **scanf()** <u>διαβάζει αλφαριθμητικά σε πίνακες χαρακτήρων</u> και το όνομα του πίνακα χωρίς δείκτη είναι η διεύθυνση του πρώτου στοιχείου του πίνακα. Έτσι για να διαβάσουμε ένα αλφαριθμητικό στον πίνακα χαρακτήρων address, θα γράψουμε
scanf("%s",address);

M.I.E. 2021-2022

Σ' αυτήν την περίπτωση, ο **address** είναι ήδη ένας δείκτης και δεν χρειάζεται να έχει μπροστά του τον τελεστή **&**.
Πρέπει να ξεχωρίζουμε τα στοιχεία των δεδομένων εισόδου χρησιμοποιώντας κενά διαστήματα, tab, ή νέες γραμμές.

Έτσι η, scanf("%d%d",&a,&b);
δέχεται μια είσοδο σαν το 10 20, αλλά δεν μπορεί να δεχτεί το 10,20. Όπως και με την **printf()**, οι κωδικοί φόρμας της **scanf()** πρέπει να βρίσκονται στην ίδια σειρά με τις μεταβλητές της λίστας ορισμάτων που δέχονται την είσοδο.

Όταν τοποθετείτε ένας αστερίσκος (*) μετά το % και πριν από τον κωδικό φόρμας, τα δεδομένα του καθοριζόμενου τύπου θα διαβαστούν, αλλά δεν θα γίνει η εκχώρηση της τιμής.

Έτσι η, scanf("%d%*c%d",&x,&y);
αν δοθεί σαν είσοδος το **10/20**, η **x** θα πάρει την τιμή 10, το σύμβολο της διαίρεσης θα απορριφθεί, και η **y** θα πάρει την τιμή 20.

Οι εντολές φόρμας μπορούν να καθορίσουν ένα μετατροπέα μεγίστου μήκους πεδίου. Αυτός ο μετατροπέας είναι ένας ακέραιος αριθμός που τοποθετείται ανάμεσα στο % και στον κωδικό της εντολής φόρμας και περιορίζει τον αριθμό των χαρακτήρων που διαβάζονται για οποιοδήποτε πεδίο. Για παράδειγμα, για να μην διαβαστούν παραπάνω από 20 χαρακτήρες στο **pedio**, θα γράφαμε, scanf("%20s",pedio);

Αν ο buffer εισόδου έχει παραπάνω από 20 χαρακτήρες, τότε η επόμενη κλήση στην είσοδο ξεκινάει από εκεί που τελειώνει αυτή η κλήση.

Για παράδειγμα, αν δώσουμε
ΑΒΓΔΕΖΗΘΙΚΛΜΝΞΟΠΡΣΤΥΦΧΨΩ
σαν απόκριση στην παραπάνω κλήση, η **scanf()** θα τοποθετήσει στο **pedio** μόνο τους πρώτους 20 χαρακτήρες, δηλαδή μέχρι και το **Y**, λόγω του καθοριστικού μέγιστου μήκους. Η συνάρτηση δεν χρησιμοποιεί ακόμη τους υπόλοιπους χαρακτήρες, ΦΧΨΩ.

Μ.Ι.Σ. 2021-2022

Αν καλέσουμε όμως άλλη μια φορά την **scanf()**
scanf("%s",nextpedio);
τότε ο υπολογιστής θα τοποθετήσει το ΦΧΨΩ στο **nextpedio**. Η είσοδος σε ένα πεδίο μπορεί να τελειώσει πριν φτάσουμε στο μέγιστο μήκος πεδίου αν ο υπολογιστής συναντήσει έναν κενό χαρακτήρα. Σ' αυτήν την περίπτωση η **scanf()** προχωράει στο επόμενο πεδίο.
Παρ' όλο που χρησιμοποιούνται κενά διαστήματα, tab, και νέες γραμμές σαν διαχωριστικά πεδίων, όταν διαβάζεται ένας χαρακτήρας, ο υπολογιστής διαβάζει όλους τους χαρακτήρες χωρίς διακρίσεις.

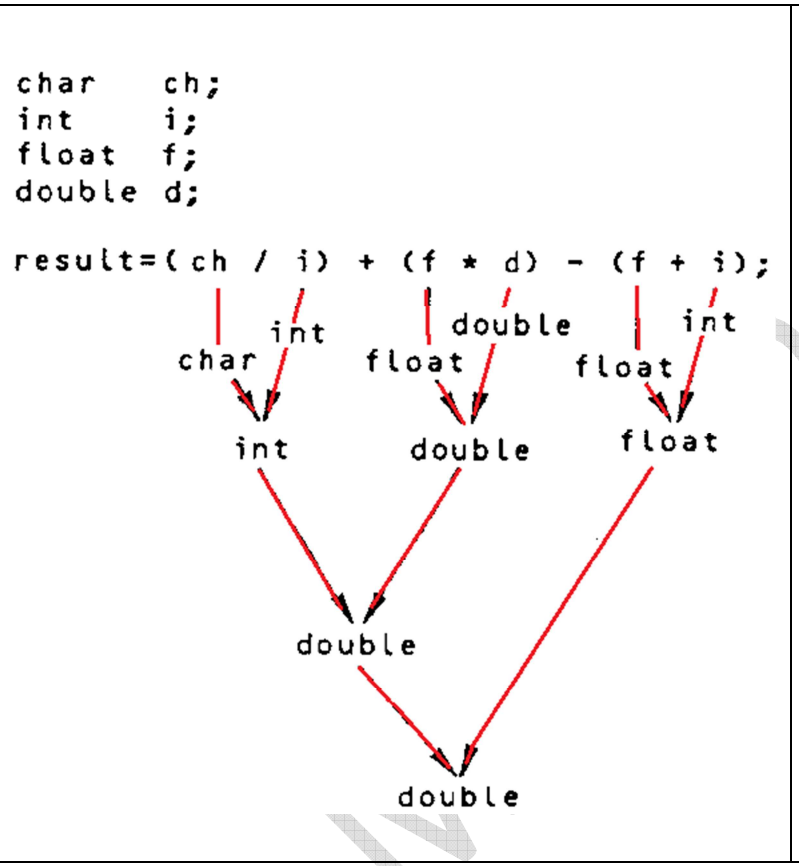
Για παράδειγμα, αν πληκτρολογήσουμε x y,
η scanf("%c%c%c",&a,&b,&c);
θα επιστρέψει τον χαρακτήρα **x** στην **a**, ένα κενό διάστημα στην **b**, και τον χαρακτήρα **y** στην **c**.
Αν στο αλφαριθμητικό ελέγχου έχουμε διάφορους χαρακτήρες, συμπεριλαμβανομένων κενών διαστημάτων, tab και νέων γραμμών, ο υπολογιστής θα χρησιμοποιήσει αυτούς τους χαρακτήρες για να εντοπίσει και να απορρίψει χαρακτήρες από τον buffer εισόδου. Ο υπολογιστής θα απορρίψει όλους τους ίδιους χαρακτήρες Για παράδειγμα, αν δοθεί σαν είσοδος το **10t20** η scanf("%st%s",&x,&y);
θα τοποθετήσει το 10 στο **x** και το 20 στο **y**. Ο υπολογιστής θα απορρίψει το χαρακτήρα **t** γιατί υπάρχει το **t** στο αλφαριθμητικό ελέγχου.

Η scanf("%s ",name);
δεν θα επιστρέψει, μέχρι να πληκτρολογήσουμε ένα χαρακτήρα αφού πληκτρολογήσουμε ένα χαρακτήρα κενού. Αυτό συμβαίνει, γιατί το κενό διάστημα μετά το **%s** λέει στην **scanf()** να διαβάζει και να απορρίπτει τα κενά διαστήματα, τα tab και τις νέες γραμμές.

Μ.Ι.Σ. 2021-2022

ΜΕΤΑΤΡΟΠΕΣ ΤΥΠΩΝ ΜΕΤΑΒΛΗΤΩΝ

Όταν έχουμε σταθερές και μεταβλητές διαφορετικών τύπων σε μια παράσταση, η C τις μετατρέπει όλες στον ίδιο τύπο. Ο μεταγλωττιστής της C θα μετατρέψει πράξη προς πράξη όλους τους τελεστέους προς τα «επάνω» στον τύπο του μεγαλύτερου τελεσταίου, όπως περιγράφεται στους παρακάτω δυο βασικούς κανόνες μετατροπής τύπων:



Όλοι οι τύποι char και short int μετατρέπονται σε int. Όλοι οι τύποι float μετατρέπονται σε double.

Για όλα τα ζευγάρια τελεστέων, η σειρά μετατροπής είναι η παρακάτω. Αν ο ένας από τους τελεστέους είναι long double, και ο άλλος τελεσταίος μετατρέπεται σε long double. Αν ο ένας τελεστέος είναι double, και ο άλλος τελεστέος μετατρέπεται σε double. Αν ο ένας από τους τελεστέους είναι long και ο άλλος τελεστέος μετατρέπεται σε long. Αν ο ένας από τους τελεστέους είναι unsigned και ο άλλος τελεστέος μετατρέπεται σε unsigned.

Αφού ο μεταγλωττιστής εφαρμόσει αυτούς τους κανόνες μετατροπής, το κάθε ζευγάρι τελεστέων θα γίνει ενός τύπου, οπότε το αποτέλεσμα της κάθε πράξης θα είναι το ίδιο με τον τύπο και των δύο τελεστέων. Για παράδειγμα, αριστερά ο υπολογιστής μετατρέπει τον ch σε int και την float f σε double. Στη συνέχεια μετατρέπεται το αποτέλεσμα ch/i σε double γιατί το f * d είναι double. Το τελικό αποτέλεσμα είναι double

IW 2021-2022

Επίπεδο	Κανόνας Μετατροπής
1	Αν ένα από τα ορίσματα είναι long double τότε και το άλλο μετατρέπεται long double.
2	διαφορετικά αν ένα από τα ορίσματα είναι double τότε και το άλλο μετατρέπεται double.
3	Αν ένα από τα ορίσματα είναι float τότε και το άλλο μετατρέπεται float.
4	διαφορετικά τα ορίσματα τύπου char και short int μετατρέπονται σε int αν μπορεί να αναπαρασταθεί η αρχική τους τιμή αλλιώς σε unsigned int.
5	Αν ένα από τα ορίσματα είναι unsigned long τότε και το άλλο μετατρέπεται unsigned long.
6	Αν τα δύο ορίσματα είναι unsigned int και long, και ο long μπορεί να αναπαραστήσει όλες τις τιμές του τύπου unsigned int τότε ο κοινός τύπος είναι ο long αλλιώς είναι ο unsigned long.
7	Αν το ένα από τα ορίσματα είναι long, μετατρέπεται και το άλλο σε long.
8	Αν το ένα από τα ορίσματα είναι unsigned int, μετατρέπεται και το άλλο σε unsigned int.
9	Φτάνοντας στο βήμα αυτό και τα δύο ορίσματα πρέπει να είναι int.

Ιεραρχικοί κανόνες μετατροπής

ΣΥΓΚΡΙΤΙΚΟΙ ΤΕΛΕΣΤΕΣ

Οι συγκριτικοί τελεστές που υποστηρίζονται από την C είναι οι,

< **μικρότερο**
> **μεγαλύτερο**
<= **μικρότερο ή ίσο**
>= **μεγαλύτερο ή ίσο**
!= **διάφορο**
= = **ίσο με**

Οι συγκριτικοί τελεστές επιδρούν στα δεδομένα όλων των βασικών τύπων. Ειδικά η σύγκριση χαρακτήρων γίνεται με βάση τον κώδικα αναπαράστασης των χαρακτήρων στην ακολουθία χαρακτήρων ASCII.

Μ.Ι.Σ. 2021-2022

ΛΟΓΙΚΕΣ ΠΡΑΞΕΙΣ

Στην C, οι λογικές πράξεις μπορούν να γίνουν μεταξύ δεδομένων λογικού τύπου (λογικών μεταβλητών και λογικών σταθερών). Οι βασικές λογικές πράξεις είναι τρείς και υλοποιούνται με τους τελεστές,

! **(άρνηση)**
&& **(σύζευξη)**
|| **(διάζευξη)**

Άρνηση
Η άρνηση μιας λογικής πρότασης είναι ΑΛΗΘΗΣ (ή ΨΕΥΔΗΣ) όταν η πρόταση είναι αντίστοιχα ΑΛΗΘΗΣ (ή ΨΕΥΔΗΣ).
Σύζευξη
Η σύζευξη δύο λογικών προτάσεων είναι ΑΛΗΘΗΣ όταν και οι δύο προτάσεις έχουν τιμή ΑΛΗΘΗΣ. Αν μία από τις προτάσεις έχει τιμή ΨΕΥΔΗΣ, τότε το αποτέλεσμα της σύζευξης έχει τιμή ΨΕΥΔΗΣ.
Διάζευξη
Η διάζευξη δύο λογικών προτάσεων έχει τιμή ΑΛΗΘΗΣ όταν μία τουλάχιστον από τις δύο προτάσεις έχει τιμή ΑΛΗΘΗΣ. Μόνο αν και οι δύο προτάσεις έχουν τιμή ΨΕΥΔΗΣ, το αποτέλεσμα της διάζευξης έχει τιμή ΨΕΥΔΗΣ.

Λογική Μεταβλητή	X	Y	! X	X && Y	X || Y
Τιμή	ΑΛΗΘΗΣ	ΑΛΗΘΗΣ	ΨΕΥΔΗΣ	ΑΛΗΘΗΣ	ΑΛΗΘΗΣ
	ΑΛΗΘΗΣ	ΨΕΥΔΗΣ	ΨΕΥΔΗΣ	ΨΕΥΔΗΣ	ΑΛΗΘΗΣ
	ΨΕΥΔΗΣ	ΑΛΗΘΗΣ	ΑΛΗΘΗΣ	ΨΕΥΔΗΣ	ΑΛΗΘΗΣ
	ΨΕΥΔΗΣ	ΨΕΥΔΗΣ	ΑΛΗΘΗΣ	ΨΕΥΔΗΣ	ΨΕΥΔΗΣ

Τονίζεται ξανά, ότι για την C η τιμή **ΑΛΗΘΗΣ** είναι οποιοσδήποτε αριθμός διάφορος του 0 ενώ η τιμή **ΨΕΥΔΗΣ** είναι ο αριθμός 0.

2021-2022

Ιεραρχία Τελεστών στις Λογικές Πράξεις

- ο τελεστής άρνησης !
- οι αριθμητικοί τελεστές *, /, %, +, -
- οι τελεστές συσχέτισης >, >=, <, <=
- οι τελεστές ισότητας ==, !=
- οι λογικοί τελεστές &&, || (οι παραστάσεις που συνδέονται με αυτούς υπολογίζονται από τα αριστερά προς τα δεξιά και ο υπολογισμός θα σταματήσει μόλις εξακριβωθεί ότι το αποτέλεσμα είναι ΑΛΗΘΕΣ ή ΨΕΥΔΕΣ).

<u>Σημείωση</u>, και όλα όταν είναι ισάξια, η επεξεργασία από την C, γίνεται από αριστερά προς τα δεξιά

Μ.Ι.Σ. 2021-2022

Η ΒΑΣΙΚΗ ΔΟΜΗ ΕΛΕΓΧΟΥ
if

Για την επίλυση πολύπλοκων προβλημάτων πολλές φορές καθίσταται αναγκαίο κάποιες εντολές του προγράμματος να μην εκτελούνται με τη σειρά που είναι γραμμένες. Η **δομή ελέγχου** είναι η δομή που χρησιμοποιείται για την επιλογή μιας επεξεργασίας μεταξύ δύο ή περισσότερων διαφορετικών διαδικασιών ανάλογα με την τιμή μιας λογικής συνθήκης.

Η δομή ελέγχου χρησιμοποιείται με δύο εκδοχές,

απλή εντολή έλεγχου
σύνθετη εντολή ελέγχου.

Η σύνταξη της απλής εντολής είναι:

```
if (συνθήκη ελέγχου)
{
        εντολή 1
        εντολή 2
        …
        εντολή ν
}
```

Τα άγκιστρα **{ }** πρέπει να υπάρχουν στην περίπτωση που η εντολή ελέγχου εκτελεί πολλές εντολές. Στην περίπτωση που η ομάδα εντολών αποτελείται από μία μόνο εντολή τότε αυτά μπορεί να παραλείπονται. Αν η συνθήκη ελέγχου έχει τιμή ΑΛΗΘΗΣ, εκτελείται η ομάδα εντολών που βρίσκεται μεταξύ του άγκιστρου έναρξης και του άγκιστρου τερματισμού. Διαφορετικά, η συγκεκριμένη ομάδα εντολών αγνοείται και ή ροή του προγράμματος μεταφέρεται στην εντολή που ακολουθεί μετά το άγκιστρο τερματισμού.

Μ.Ι.Σ. 2021-2022

Η εντολή **if - else** αναφέρεται και ως σύνθετη εντολή ελέγχου. Με την εντολή αυτή επιλέγεται μία από τις δύο επεξεργασίες που μπορούν να γίνουν, ανάλογα με την τιμή μιας συνθήκης ελέγχου.

if *(συνθήκη ελέγχου)*
    ομάδα εντολών 1
else
    ομάδα εντολών 2

Όταν οι εναλλακτικές περιπτώσεις είναι περισσότερες από δύο, τότε μπορούν να χρησιμοποιηθούν πολλές εντολές if κάθε μία μέσα στην άλλη, που ονομάζονται **εμφωλιασμένες *δομές ελέγχου***,

if (συνθήκη ελέγχου 1)
    ομάδα εντολών 1
else
    if (συνθήκη ελέγχου 2)
        ομάδα εντολών 2
        …
    else
        ομάδα εντολών ν

Όταν κάποια από τις συνθήκες ελέγχου είναι αληθής, εκτελούνται οι εντολές που βρίσκονται στο αντίστοιχο τμήμα. Κάθε τμήμα της δομής ελέγχου μπορεί να περιέχει οποιεσδήποτε εντολές. Συνεπώς μπορεί να περιλαμβάνει και άλλες δομές ελέγχου.

Μ.Π.Σ. 2021-2022

ΠΑΡΑΣΤΑΣΕΙΣ ΣΕ ΣΥΝΘΗΚΗ

Υπάρχει στην C μια ειδική χρήση του τελεστή υπό συνθήκη,

*συνθήκη ελέγχου **?** παράσταση 1 **:** παράσταση 2*

Εάν η συνθήκη ελέγχου είναι αληθής (όχι μηδέν) τότε όλη η παράσταση υπό συνθήκη έχει την ίδια τιμή με την *παράσταση 1*.
Εάν η συνθήκη ελέγχου είναι ψευδής (μηδέν), τότε ολόκληρη η παράσταση υπό συνθήκη έχει την ίδια τιμή με την *παράσταση 2*.
Μπορούμε να χρησιμοποιήσουμε την παράσταση υπό συνθήκη όταν έχουμε μια μεταβλητή, στην οποία μπορούν να καταχωρηθούν δύο πιθανές τιμές.

Μια πρόταση **if - else** μπορεί να πετύχει το ίδιο αποτέλεσμα, μ' αυτό του τελεστή υπό συνθήκη. Όμως οι παραστάσεις υπό συνθήκη είναι μικρές απλές και οδηγούν σε γρήγορο κώδικα γλώσσας μηχανής.

```
#include <stdio.h>
int main()
{
        int x, y;
        printf("Δώσε έναν ακέραιο αριθμό : ");
        scanf("%d",&y);

        x = (y > 0) ? y : -y;

        printf("Η απόλυτη τιμή του αριθμού %d είναι %d\n",y,x);
```

Μ.Ι.Σ. 2021-2022

```
        return 0;
}
```

Το παραπάνω παράδειγμα υπολογίζει την απόλυτη τιμή ενός αριθμού y. Αυτό γίνεται με τη χρήση της παράστασης υπό συνθήκη **(y > 0) ? y : -y**.

Εάν το y είναι μεγαλύτερο του μηδενός τότε **x = y**, αλλιώς **x = -y**.

Αντίστοιχα η παράσταση υπό συνθήκη μπορεί να αντικατασταθεί με την εντολή **if …else**.

```
if (y > 0)
        x = y;
else
        x = -y;
```

Μ.Ι.Σ. 2021-2022

ΠΟΛΛΑΠΛΕΣ ΣΥΝΘΗΚΕΣ ΜΕ ΤΗΝ ΕΝΤΟΛΗ switch

Με την δομή **if – else… if – else… if – else…** μπορούμε να γράψουμε προγράμματα με πολλά εμφωλιασμένα **if** αλλά είναι αντιληπτό ότι αυτό δεν είναι λειτουργικό σε σύνθετα **if**

Στην περίπτωση αυτή η C διαθέτει την εντολή **switch**,

```
switch (επιλογέας)
{
        case τιμή 1 :
                ομάδα εντολών 1;
                break;
        case τιμή 2 :
                ομάδα εντολών 2;
                break;
        …
        case τιμή N :
                ομάδα_εντολών N;
                break;
        default :
                ομάδα εντολών default;
                break;
}
```

Μ.Ι.Σ. 2021-2022

Παράδειγμα **switch** με αριθμό,

```
int num;
printf("Δώσε έναν αριθμό : ");
scanf("%d",&num);
switch (num % 2)
{
        case 0 :
                printf("Ο αριθμός είναι άρτιος.\n");
                break;
        case 1 :
                printf("Ο αριθμός είναι περιττός.\n");
                break;
}
```

Μ.Ι.Σ. 2021-2022

Παράδειγμα **switch** με χαρακτήρα,

```
char ch;

printf("a. Εισαγωγή\n");
printf("β. Διόρθωση\n");

printf("Δώστε επιλογή (α – β) : ");

scanf("%c",&ch);

switch (ch)
{
        case 'α' :
        case 'A':
                printf("Επέλεξες το α ή A.\n");
                break;
        case 'β' :
        case 'B' :
                printf("Επέλεξες το β ή B.\n");
                break;
        default :
                printf("Έκανες λάθος επιλογή.\n");
                break;
}
```

Μ.Ι.Σ. 2021-2022

ΟΙ ΜΕΤΑΒΛΗΤΕΣ ΤΥΠΟΥ

char

Ένας τύπος δεδομένων στην C είναι το **char** character/χαρακτήρας. Ένας χαρακτήρας είναι μια τιμή ενός byte την οποία μπορούμε να χρησιμοποιήσουμε για να κρατήσουμε εκτυπώσιμους χαρακτήρες ή ακεραίους από -128 έως 127.

Ειδικά μια σταθερά (σαν χαρακτήρας) θα περιβάλλεται από μονά εισαγωγικά.

```
#include <stdio.h>
int main()
{
        char mychaaa;
                do {
                        printf("Δώσε χαρακτήρα (q ή Q για έξοδος) : ");

                        scanf("%c", &mychaa);

                        printf("Ο χαρακτήρας ήταν το %c\n",mychaa);

                } while (mychaa != 'q' && mychaa != 'Q');

        return 0;
}
```

Με την **scanf()** μπορούμε για να διαβάσουμε ένα χαρακτήρα από το πληκτρολόγιο, αλλά μια άλλη μέθοδος είναι να χρησιμοποιούμε τη συνάρτηση βιβλιοθήκης **getchar().**

Μ.Ι.Σ. 2021-2022

A → Ο χαρακτήρας 'A' μετατρέπεται στον ASCII κώδικά

↓

0	**1**	**0**	**0**	**0**	**0**	**0**	**1**

Ο ASCII κώδικάς του αποθηκεύεται σε δυαδική μορφή

%c → Το περιεχόμενο εμφανίζεται σαν χαρακτήρας A

%d → Το περιεχόμενο εμφανίζεται σαν δεκαδικός αριθμός 65